

PROJECT REPORT

1.cover page

STUDENT TO-DO LIST MANAGER

Project Report

Submitted by:

Name: Prachi Singh

Registration No.: 26BCE11448

Course/Class: B.Tech CS Core

College/School: VIT BHOPAL(School-Scope)

Technology Used: Python, Tkinter, JSON, Git & GitHub

Academic Year: 2026–27

2.Add the report section

1. Introduction

Explain what a To-Do List is and why students need one.

2. Problem Statement

Explain the problem of managing assignments, deadlines, projects and daily activities.

3. Objectives

For example:

- To create a simple task-management application.
- To help students organize their tasks.
- To track task completion.

- To identify high-priority and overdue tasks.
- To provide search and filtering facilities.

4. Proposed Solution

The proposed solution is a Student To-Do List Management System designed to help students organize and manage their daily academic and personal tasks efficiently.

The system provides a simple and user-friendly interface where users can add, view, update, complete, and delete tasks. Each task can contain details such as the task title, description, priority, and deadline. Completed tasks can be marked separately so that users can easily track their progress.

The proposed system aims to replace traditional methods such as notebooks, sticky notes, and scattered reminders with a centralized digital task-management application. It helps users prioritize their work, avoid missing deadlines, and maintain a clear overview of pending and completed activities.

5. Requirements

Hardware:

- Computer/laptop
- Minimum 4 GB RAM
- Keyboard and mouse

Software:

- Python 3
- Tkinter
- Git
- GitHub
- VS Code/PyCharm/any Python IDE

6. Functional Requirements

Include:

- Add task
- Update task
- Delete task
- Complete task
- Search task
- Filter task
- Set priority
- Set due date
- View progress/analytics
- Save tasks

7. Non-Functional Requirements

Include at least four:

- **Usability:** Interface should be simple and easy to understand.

- **Reliability:** Tasks should be saved correctly.
- **Performance:** The application should respond quickly.
- **Maintainability:** Code should be divided into separate modules.
- **Security:** User data should remain locally stored.

7. Design Diagrams

1. **Use Case Diagram** – shows what the student/user can do with the application.
2. **Workflow Diagram** – shows the step-by-step flow of the application.
3. **Sequence Diagram** – shows how the user, GUI, task manager, and storage interact.
4. **Class/Component Diagram** – shows the different Python modules and their relationships.
5. **ER Diagram** – shows the structure of your task data because your project uses JSON storage.

8. Design Decisions & Rationale

For the Student To-Do List Manager, the following design decisions were made:

Python: Python was selected because it is easy to understand, has a large number of libraries, and is suitable for developing a student-level desktop application.

Tkinter: Tkinter was selected for creating the graphical user interface because it is included with Python and provides the required GUI components without requiring a complex framework.

JSON Storage: JSON was selected for local storage because it is lightweight, human-readable and easy to process using Python.

Modular Architecture: The application is divided into different modules such as task management, validation, storage, search and analytics. This makes the code easier to understand, test and maintain.

Single-user Design: The application is designed for individual students rather than multi-user collaboration, keeping the application simple and suitable for personal task management.

9. Implementation Details

The Student To-Do List Manager is implemented using Python.

The major components are:

- **Task Model:** Stores task information such as ID, title, description, due date, priority, category and status.
- **Task Manager:** Handles adding, updating, completing and deleting tasks.
- **Validation:** Checks whether the entered task information is valid.
- **Storage:** Saves and loads task information using JSON.
- **Search:** Allows users to find particular tasks.

- **Analytics:** Calculates information such as completion rate and high-priority tasks.
- **GUI:** Provides an interactive interface using Tkinter.

10 Screenshots / Results

- Add screenshots of your **actual working application** here.
- Recommended screenshots:
 - **Figure 1 – Main Interface**
 - Screenshot showing the main To-Do List window.
 - **Figure 2 – Adding a Task**
 - Screenshot showing a task being entered with title, description, date, priority and category.
 - **Figure 3 – Task List**
 - Screenshot showing multiple tasks.
 - **Figure 4 – Search/Filter**
 - Screenshot demonstrating the search or filtering feature.
 - **Figure 5 – Completed Task**
 - Screenshot showing a task marked as completed.
 - **Figure 6 – Analytics**
 - Screenshot showing completion rate or task statistics.

11. Testing Approach

Testing was performed to verify that the major functions of the application work correctly.

Test Case	Action/Input	Expected Result
TC01	Add a valid task	Task is added successfully
TC02	Enter task details	Details are stored correctly
TC03	Complete a task	Task status changes to Completed
TC04	Delete a task	Selected task is removed
TC05	Search for a task	Matching task is displayed
TC06	Filter by priority	Tasks with selected priority are displayed
TC07	Restart application	Previously saved tasks are loaded
TC08	Enter invalid data	Validation prevents incorrect input
TC09	Check analytics	Correct task statistics are displayed

You can add a **Result** column after testing:

Test Case	Expected Result	Actual Result	Status
TC01	Task added	Task added	Pass
TC02	Details saved	Details saved	Pass
TC03	Status changes	Status changes	Pass

Test Case	Expected Result	Actual Result	Status
-----------	-----------------	---------------	--------

TC04	Task deleted	Task deleted	Pass
------	--------------	--------------	------

Only use **Pass** after you actually test each feature.

12. Challenges Faced

During development of the Student To-Do List Manager, several challenges were encountered.

- Designing a simple and user-friendly graphical interface.
- Managing task information efficiently.
- Implementing persistent storage so that tasks remain available after restarting the application.
- Validating user inputs.
- Implementing search and filtering functionality.
- Calculating task completion statistics.
- Organizing the program into separate modules.
- Debugging errors during development.
- Managing the project using Git and GitHub.

These challenges helped improve understanding of Python programming, GUI development, data storage and software development practices.

13. Learnings & Key Takeaways

The project provided practical experience in developing a complete software application.

The major learnings include:

- Understanding Python programming concepts.
- Creating a graphical user interface using Tkinter.
- Using JSON for data storage.
- Applying modular programming.
- Implementing validation and error handling.
- Creating search and filtering functionality.
- Writing basic automated tests.
- Understanding software design diagrams.
- Using Git for version control.
- Using GitHub to store and manage a project.
- Understanding the complete development process from problem identification to implementation and testing.

14. Future Enhancements

The project can be improved further by adding:

1. **Calendar View** – Display tasks according to their due dates.
2. **Reminder Notifications** – Notify users about upcoming deadlines.

3. **CSV/PDF Export** – Allow users to export their task lists.
 4. **User Login** – Provide individual accounts for different users.
 5. **Cloud Synchronization** – Store tasks online and access them from different devices.
 6. **Dark Mode** – Provide an alternative interface theme.
 7. **Recurring Tasks** – Allow users to create daily, weekly or monthly tasks.
 8. **Task Sorting** – Sort tasks by deadline, priority or category.
-

15. References

You can include:

1. Python Documentation – Python programming language documentation.
2. Tkinter Documentation – Documentation related to Python GUI development.
3. Git Documentation – Reference for Git version control.
4. GitHub Documentation – Reference for repository management and collaboration.
5. Vityarthi – Project guidelines.
6. Official documentation/resources used during development of the project.

